Assignment-4

Web Programming using

Python and JavaScript

Submitted By:

Name: Aditya Pal

Course: B.Tech CSE

Year: 2nd Year, 4th Semester

Roll No: 2401010119

Submitted To:

Name: Dr. Mansi Kajal

❖ **AIM:** To build a dynamic To-Do List web application using HTML, CSS, JavaScript and integrate a database-backed backend to store tasks permanently (CRUD).

❖ **TASKS:**

▪ **Task 1: Project Setup & Introduction -**

• Create a project folder named dynamic_todo_list

• Inside the folder, create:

o index.html

o style.css

o script.js

• Add header comments in HTML (project title, name, date)

• Display a short description of the to-do list application

Deliverable:

Project folder with HTML, CSS, and JS files linked correctly.

▪ **Task 2: To-Do List Interface (HTML) -**

• Create an input field for entering tasks

• Add an “Add Task” button

• Display tasks in a list format (<ul> or <ol>)

Deliverable:

Basic to-do list layout created using HTML.

- **Task 3: Styling the To-Do List (CSS) -**

• Apply CSS for:

o Input box and buttons

o Task list alignment

o Visual separation of tasks

• Ensure the interface is clean and readable

Deliverable:

style.css file with neat and consistent styling.

- **Task 4: Add & Display Tasks (JavaScript)** -

• Capture user input using JavaScript

• Add new tasks dynamically to the list

• Prevent empty tasks from being added

Deliverable:

Functionality to add and display tasks dynamically.

- **Task 5: Edit & Delete Tasks -**

• Add “Edit” and “Delete” buttons for each task

• Allow users to modify task text

• Remove tasks from the list dynamically

Deliverable:

Fully functional edit and delete features.

- **Task 6 (Bonus): Task Completion Status -**

• Add a checkbox or toggle for task completion

• Visually indicate completed tasks (e.g., strikethrough)

• Optional: Count completed vs pending tasks

Deliverable:

Enhanced to-do list with task completion tracking.

❖ CODES:

➢ HTML Code(index.html):

```
<!--
Project Title: Advanced Dynamic To-Do List
Student Name: Aditya Pal
Date: 13 February 2026
Description: A modern, interactive to-do list with local storage, progress
tracking, edit, delete, and completion features.
-->

<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Smart To-Do List</title>
    <link rel="stylesheet" href="style.css">
</head>
<body>

<div class="app-container">

    <h1>✨ To-Do List</h1>
    <p class="subtitle">Stay organized. Stay productive.</p>

    <!-- Input Section -->
    <div class="input-group">
        <input type="text" id="taskInput" placeholder="What needs to be
done?">
        <button id="addTaskBtn">+</button>
```

```
    </div>

    <!-- Task Stats -->
    <div class="stats">
        <span id="taskCount">0 Tasks</span>
        <span id="completedCount">0 Completed</span>
    </div>

    <!-- Progress Bar -->
    <div class="progress-container">
        <div id="progressBar"></div>
    </div>

    <!-- Task List -->
    <ul id="taskList"></ul>

    <!-- Clear Button -->
    <button id="clearAllBtn" class="clear-btn">Clear All</button>

</div>

<script src="script.js"></script>
</body>
</html>
```

➢ CSS Code(style.css):

```
/* Reset */
* {
    box-sizing: border-box;
}

body {
    margin: 0;
    font-family: "Segoe UI", Tahoma, sans-serif;
    background-color: #f4f6f8;
    display: flex;
    justify-content: center;
    padding: 40px 15px;
}

/* Main Container */
.app-container {
    width: 100%;
    max-width: 520px;
    background-color: #ffffff;
    padding: 30px;
    border-radius: 8px;
    box-shadow: 0 4px 12px rgba(0, 0, 0, 0.08);
```

```
}

/* Title */
h1 {
    margin: 0;
    font-size: 24px;
    font-weight: 600;
    color: #1f2937;
    text-align: center;
}

.subtitle {
    text-align: center;
    font-size: 14px;
    color: #6b7280;
    margin: 8px 0 25px;
}

/* Input Section */
.input-group {
    display: flex;
}

#taskInput {
    flex: 1;
    padding: 10px 12px;
    border: 1px solid #d1d5db;
    border-radius: 6px 0 0 6px;
    font-size: 14px;
    outline: none;
}

#taskInput:focus {
    border-color: #2563eb;
}

#addTaskBtn {
    width: 50px;
    border: none;
    background-color: #2563eb;
    color: white;
    font-size: 18px;
    border-radius: 0 6px 6px 0;
    cursor: pointer;
    transition: background 0.2s ease;
}

#addTaskBtn:hover {
```

```
    background-color: #1e4fd8;
}

/* Stats */
.stats {
    display: flex;
    justify-content: space-between;
    margin: 18px 0 8px;
    font-size: 13px;
    color: #4b5563;
}

/* Progress Bar */
.progress-container {
    width: 100%;
    height: 6px;
    background-color: #e5e7eb;
    border-radius: 4px;
    margin-bottom: 20px;
}

#progressBar {
    height: 100%;
    width: 0%;
    background-color: #2563eb;
    border-radius: 4px;
    transition: width 0.3s ease;
}

/* Task List */
ul {
    list-style: none;
    padding: 0;
    margin: 0;
}

li {
    display: flex;
    align-items: center;
    justify-content: space-between;
    background-color: #f9fafb;
    padding: 12px;
    margin-bottom: 10px;
    border: 1px solid #e5e7eb;
    border-radius: 6px;
    transition: background 0.2s ease;
}
```

```css
li:hover {
    background-color: #f3f4f6;
}

/* Task Info */
.task-info {
    flex: 1;
    margin-left: 10px;
    font-size: 14px;
    color: #111827;
}

.task-info small {
    color: #6b7280;
    font-size: 12px;
}

/* Completed */
.completed {
    text-decoration: line-through;
    color: #9ca3af;
}

/* Action Buttons */
.action-btn {
    border: none;
    padding: 6px 10px;
    margin-left: 6px;
    border-radius: 4px;
    font-size: 12px;
    cursor: pointer;
    transition: background 0.2s ease;
}

/* Edit Button */
.edit {
    background-color: #e5e7eb;
    color: #111827;
}

.edit:hover {
    background-color: #d1d5db;
}

/* Delete Button */
.delete {
    background-color: #ef4444;
    color: white;
```

```
}

.delete:hover {
    background-color: #dc2626;
}

/* Clear All Button */
.clear-btn {
    width: 100%;
    margin-top: 18px;
    padding: 10px;
    border: none;
    border-radius: 6px;
    background-color: #374151;
    color: white;
    font-size: 14px;
    cursor: pointer;
    transition: background 0.2s ease;
}

.clear-btn:hover {
    background-color: #1f2937;
}

/* Responsive */
@media (max-width: 480px) {
    .app-container {
        padding: 20px;
    }

    h1 {
        font-size: 20px;
    }
}
```

➢ JAVASCRIPT Code(script.js):

```
const taskInput = document.getElementById("taskInput");
const addTaskBtn = document.getElementById("addTaskBtn");
const taskList = document.getElementById("taskList");
const taskCount = document.getElementById("taskCount");
const completedCount = document.getElementById("completedCount");
const progressBar = document.getElementById("progressBar");
const clearAllBtn = document.getElementById("clearAllBtn");

let tasks = JSON.parse(localStorage.getItem("tasks")) || [];

// Add Task
function addTask(text) {
```

```
    const task = {
        id: Date.now(),
        text: text,
        completed: false,
        date: new Date().toLocaleString()
    };
    tasks.push(task);
    saveTasks();
    renderTasks();
}

// Render Tasks
function renderTasks() {
    taskList.innerHTML = "";
    let completedTasks = 0;

    tasks.forEach(task => {
        const li = document.createElement("li");

        const checkbox = document.createElement("input");
        checkbox.type = "checkbox";
        checkbox.checked = task.completed;

        const div = document.createElement("div");
        div.className = "task-info";
        div.innerHTML =
`<strong>${task.text}</strong><br><small>${task.date}</small>`;

        if (task.completed) {
            div.classList.add("completed");
            completedTasks++;
        }

        checkbox.addEventListener("change", () => {
            task.completed = checkbox.checked;
            saveTasks();
            renderTasks();
        });

        const editBtn = document.createElement("button");
        editBtn.textContent = "Edit";
        editBtn.className = "action-btn edit";
        editBtn.onclick = () => {
            const newText = prompt("Edit task:", task.text);
            if (newText && newText.trim() !== "") {
                task.text = newText.trim();
                saveTasks();
                renderTasks();
```

```
            }
        };

        const deleteBtn = document.createElement("button");
        deleteBtn.textContent = "Delete";
        deleteBtn.className = "action-btn delete";
        deleteBtn.onclick = () => {
            tasks = tasks.filter(t => t.id !== task.id);
            saveTasks();
            renderTasks();
        };

        li.appendChild(checkbox);
        li.appendChild(div);
        li.appendChild(editBtn);
        li.appendChild(deleteBtn);

        taskList.appendChild(li);
    });

    updateStats(completedTasks);
}

// Update Stats & Progress
function updateStats(completedTasks) {
    taskCount.textContent = `${tasks.length} Tasks`;
    completedCount.textContent = `${completedTasks} Completed`;

    const percent = tasks.length === 0 ? 0 :
        (completedTasks / tasks.length) * 100;
    progressBar.style.width = percent + "%";
}

// Save to localStorage
function saveTasks() {
    localStorage.setItem("tasks", JSON.stringify(tasks));
}

// Event Listeners
addTaskBtn.addEventListener("click", () => {
    const text = taskInput.value.trim();
    if (text !== "") {
        addTask(text);
        taskInput.value = "";
    }
});

// Add with Enter key
```

```
taskInput.addEventListener("keypress", (e) => {
    if (e.key === "Enter") {
        addTaskBtn.click();
    }
});

// Clear All
clearAllBtn.addEventListener("click", () => {
    if (confirm("Are you sure you want to delete all tasks?"))  {
        tasks =  [];
        saveTasks();
        renderTasks();
    }
});

// Initial Render
```

➢ OUTPUT: